

Assignment #1: 虚拟机, Shell & 大语言模型

Updated 2309 GMT+8 Feb 20, 2025

2025 spring, Compiled by 李皓翔 物院

作业的各项评分细则及对应的得分

标准	等级	得分
按时提交	完全按时提交: 1分 提交有请假说明: 0.5分 未提交: 0分	1分
源码、耗时 (可选)、解题思路 (可选)	提交了4个或更多题目且包含所有必要信息: 1分 提交了2个或以上题目但不足4个: 0.5分 少于2个: 0分	1分
AC代码截图	提交了4个或更多题目且包含所有必要信息: 1分 提交了2个或以上题目但不足4个: 0.5分 少于: 0分	1分
清晰头像、PDF文件、MD/DOC附件	包含清晰的Canvas头像、PDF文件以及MD或DOC格式的附件: 1分 缺少上述三项中的任意一项: 0.5分 缺失两项或以上: 0分	1分
学习总结和个人收获	提交了学习总结和个人收获: 1分 未提交学习总结或内容不详: 0分	1分
总得分: 5	总分满分: 5分	

说明:

1. 解题与记录:

◦ 对于每一个题目, 请提供其解题思路 (可选), 并附上使用Python或C++编写的源代码 (确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted)。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。(推荐使用Typora <https://typoraio.cn> 进行编辑, 当然你也可以选择Word。)无论题目是否已通过, 请标明每个题目大致花费的时间。

2. 课程平台与提交安排:

• 我们的课程网站位于Canvas平台 (<https://pku.instructure.com>)。该平台将在第2周选课结束后正式启用。在平台启用前, 请先完成作业并将作业妥善保存。待Canvas平台激活后, 再上传你的作业。

- 提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. 延迟提交:

- 如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

27653: Fraction类

<http://cs101.openjudge.cn/practice/27653/>

思路:

代码:

```
import math

class fraction:
    def __init__(self, up, down):
        self.up = up
        self.down = down
        self.simplify()

    def __str__(self):
        return f"{self.up}/{self.down}"

    def __add__(self, other):
        new_up = self.up * other.down + other.up * self.down
        new_down = self.down * other.down
        return fraction(new_up, new_down)

    def __sub__(self, other):
        new_up = self.up * other.down - other.up * self.down
        new_down = self.down * other.down
        return fraction(new_up, new_down)

    def __mul__(self, other):
        new_up = self.up * other.up
        new_down = self.down * other.down
        return fraction(new_up, new_down)

    def __truediv__(self, other):
        new_up = self.up * other.down
        new_down = self.down * other.up
        return fraction(new_up, new_down)
```

```
def simplify(self):
    gcd = math.gcd(self.up, self.down)
    self.up //= gcd
    self.down //= gcd

a,b,c,d=map(int,input().split())
x=fraction(a,b)
y=fraction(c,d)
print(x+y)
```

代码运行截图 (至少包含有"Accepted")

27653:Fraction类

查看

提交

统计

提问

总时间限制: 1000ms 内存限制: 65536kB

描述

实现用户定义的类，一个常用的例子是构建实现抽象数据类型Fraction的类。我们已经看到，Python提供了很多数值类。但是在有些时候，需要创建“看上去很像”分数的数据对象。

像 3/5 这样的分数由两部分组成。斜线左侧的值称作分子，可以是任意整数。斜线右侧的值称作分母，可以是任意大于0的整数（负的分数的分子带负号）。尽管可以用浮点数来近似表示分数，但我们在此希望能精确表示分数的值。

Python 3.0 对分数的支持... 这里我们使用 Python 3.0 的 Fraction 类...

全局题号 27653

添加于 2024-02-25

提交次数 150

尝试人数 101

通过人数 100

你的提交记录

#	结果	时间
1	Accepted	2025-02-21

1760.袋子里最少数目的球

<https://leetcode.cn/problems/minimum-limit-of-balls-in-a-bag/>

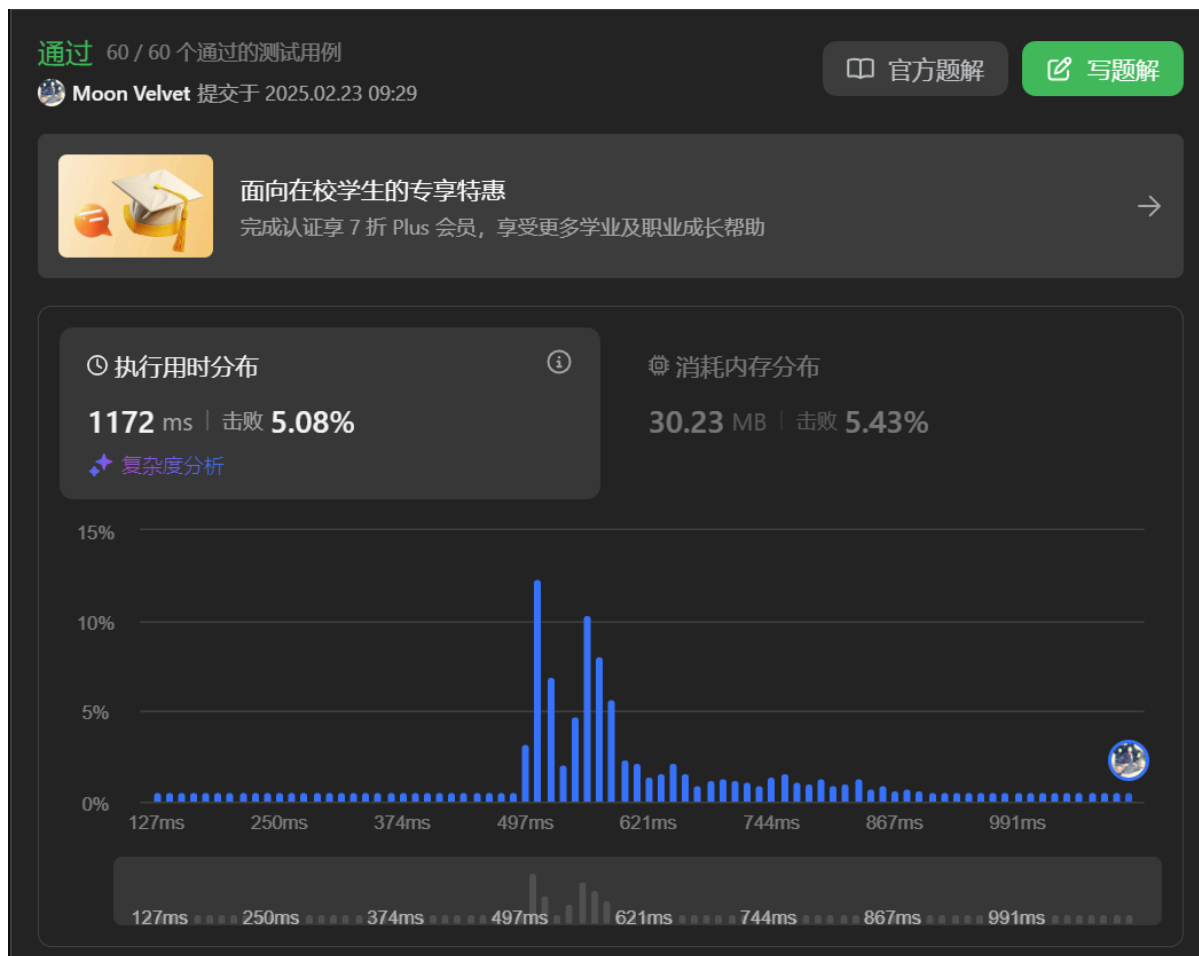
思路：

类比河中跳石头

代码：

```
class Solution:
    def minimumSize(self, nums: List[int], maxOperations: int) -> int:
        def judge(ans):
            return sum(list(map(lambda x: (x-1)//ans+1, nums)))
        left = len(nums) + maxOperations
        right = 1
        while left < right:
            mid = (left + right) // 2
            if judge(mid) <= maxOperations:
                right = mid
            else:
                left = mid + 1
        return left
```

代码运行截图 (至少包含有"Accepted")



04135: 月度开销

<http://cs101.openjudge.cn/practice/04135>

思路:

代码:

```
N,M=map(int,input().split())
it=[int(input()) for _ in range(N)]
l=max(it)
r=sum(it)+1
k=0
while l<r :
    p=(l+r)//2
    s=1
    su=0
    for i in it:
        if su+i>p:
            su=i
            s+=1
        else:
            su+=i
```

```
if s>M:
    l=p+1
else:
    r=p
print(l)
```

代码运行截图 (至少包含有"Accepted")

#48395953提交状态

[查看](#) [提交](#) [统计](#) [提问](#)

状态: Accepted

源代码

```
N,M=map(int,input().split())
it=[int(input()) for _ in range(N)]
l=max(it)
r=sum(it)+1
k=0
while l<r :
    p=(l+r)//2
    s=1
    su=0
    for i in it:
        if su+i>p:
            su=i
            s+=1
        else:
            su+=i
    if s>M:
        l=p+1
    else:
        r=p
print(l)
```

基本信息

#: 48395953
题目: 04135
提交人: 24n2400011450
内存: 7568kB
时间: 567ms
语言: Python3
提交时间: 2025-03-01 09:25:55

©2002-2022 POJ 京ICP备20010980号-1

[English](#) [帮助](#) [关于](#)

27300: 模型整理

<http://cs101.openjudge.cn/practice/27300/>

思路:

代码:

```
from collections import defaultdict

def count(m):
    dic={'B':1000,'M':1}
    return dic[m[-1]]*float(m[:-1])

n=int(input())
l=defaultdict(list)
for i in range(n):
    a,b=map(str,input().split('-'))
    l[a].append(b)

out=[]
for i in l:
```

```

        out.append(i)
    m=len(out)
    for i in range(m):
        for j in range(m-i-1):
            if out[j]>out[j+1]:
                out[j],out[j+1]=out[j+1],out[j]
    for i in out:
        inner=l[i]
        m=len(inner)
        for k in range(m):
            for j in range(m-k-1):
                if count(inner[j])>count(inner[j+1]):
                    inner[j],inner[j+1]=inner[j+1],inner[j]
    print(i+' ': '+', '.join(inner))

```

代码运行截图 (至少包含有"Accepted")

#48396487提交状态

状态: Accepted

源代码

```

from collections import defaultdict

def count(m):
    dic={'B':1000,'M':1}
    return dic[m[-1]]*float(m[:-1])

n=int(input())
l=defaultdict(list)
for i in range(n):
    a,b=map(str,input().split('-'))
    l[a].append(b)

out=[]
for i in l:
    out.append(i)
m=len(out)
for i in range(m):
    for j in range(m-i-1):
        if out[j]>out[j+1]:
            out[j],out[j+1]=out[j+1],out[j]
for i in out:
    inner=l[i]
    m=len(inner)
    for k in range(m):
        for j in range(m-k-1):
            if count(inner[j])>count(inner[j+1]):
                inner[j],inner[j+1]=inner[j+1],inner[j]
    print(i+' ': '+', '.join(inner))

```

Q5. 大语言模型（LLM）部署与测试

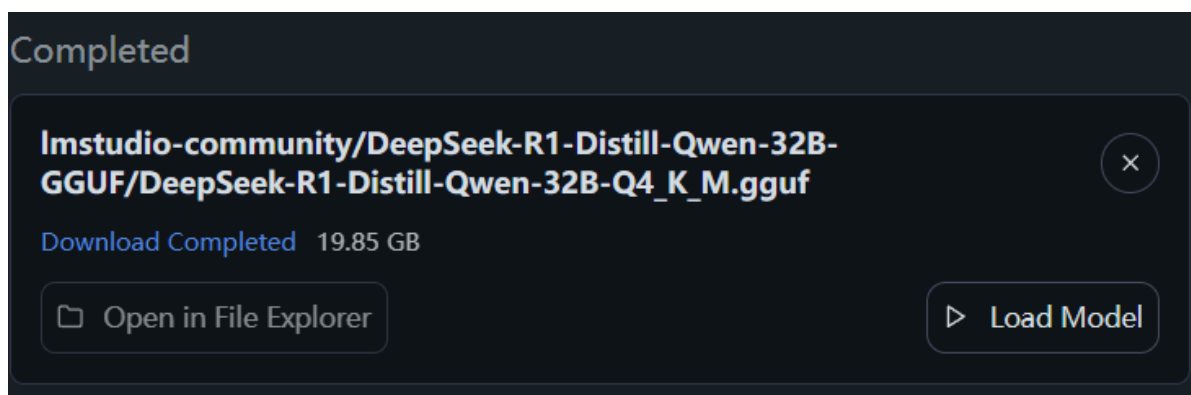
本任务旨在本地环境或通过云虚拟机（如 <https://clab.pku.edu.cn/> 提供的资源）部署大语言模型（LLM）并进行测试。用户界面方面，可以选择使用图形界面工具如 <https://lmstudio.ai> 或命令行界面如 <https://www.ollama.com> 来完成部署工作。

测试内容包括选择若干编程题目，确保这些题目能够在所部署的LLM上得到正确解答，并通过所有相关的测试用例（即状态为Accepted）。选题应来源于在线判题平台，例如 OpenJudge、Codeforces、LeetCode 或洛谷等，同时需注意避免与已找到的AI接受题目重复。已有的AI接受题目列表可参考以下链接：

https://github.com/GMyhf/2025spring-cs201/blob/main/AI_accepted_locally.md

请提供你的最新进展情况，包括任何关键步骤的截图以及遇到的问题和解决方案。这将有助于全面了解项目的推进状态，并为进一步的工作提供参考。

使用LM Studio，本地下载部署了Deepseek r1



本机为ThinkPad T14p Gen 2，CPU 为 intel vPro i9，内存32GB，GPU为 RTX 3050，带此模型略显吃力，测试了leetcode上的随机热题100中的一道：

嗯，我现在要解决这个问题。题目是说给一个整数数组prices，其中每个元素代表某只股票在第i天的价格。我可以在每一天选择买或者卖股票，但任何时候最多只能持有一股。也就是说，不能同时持有多个股票。而且，我可以同一天买卖。目标是要找到我能获得的最大利润。

那我得想一下怎么计算这个最大利润。记得之前好像有一个类似的问题，比如“最佳时机”或者类似的题目，我记得可能需要找出所有可以盈利的区间，并加起来。比如，如果价格一直上涨，那么最大的利润就是从最低点买入，最高点卖出。但是如果中间有波动的话，就要选择在每一个上升波段的时候买进然后卖出。

那我该怎么处理这些情况呢？比如说，在示例1中，prices是7,1,5,3,6,4。这个时候，最优的方式是在第2天（价格1）买入，第三天卖出，赚了4；接着第四天买入，第五天卖出，赚了3，总利润7。

那这样的策略是不是应该寻找所有相邻两天的正差价，并把它们加起来？比如，在示例1中，第二天到第三天是+4，第三天到第四天是-2（不计），第四天到第五天是+3，而第五天到第六天是-2。那么总利润是4+3=7。

那这样的话，只要我们能够找到所有连续上涨的日子，并把它们的差价加起来，就能得到最大利润了。这可能吗？

那比如说，在示例2中，价格一直上涨，所以我们只需要计算最后一天和第一天之间的差价。或者，是否等同于将每一天的正差价相加？比如1到2是+1，2到3是+1，3到4是+1，4到5是+1，这样总共有四个+1，合计4，这正好是示例的输出。

那这样的话，解决方法可能就是遍历价格数组，找出所有相邻两天的正差价，并将它们加起来。这就是最大的利润了，对吗？

🔄 重新加载上次使用的模型 (Ctrl +R)



模型直接给出了最佳贪心算法，且思考过程清晰

为了解决这个问题，我们需要找到在给定股票价格数组中可以获得的最大利润。每一天可以买入或卖出股票，但任何时候最多只能持有一股。我们的目标是最大化交易收益。

方法思路

我们可以通过遍历股票价格数组，计算所有相邻天数的正差价并累加来得到最大利润。这种方法的核心思想是：在每一个上涨区间进行买卖操作，从而赚取每一段的利润之和。

具体步骤如下：

1. 初始化一个变量 `max_profit` 为0，用于记录累计的最大利润。
2. 遍历价格数组，从第2天开始计算每一天与前一天的价格差值。
3. 如果当前天数的价格高于前一天，则将它们的价格差加到 `max_profit` 中。
4. 遍历结束后返回 `max_profit` 作为结果。

这种方法的时间复杂度为 $O(n)$ ，其中 n 是价格数组的长度，因为我们只需遍历数组一次。

解决代码

```
def maxProfit(prices):  
    max_profit = 0  
    for i in range(1, len(prices)):  
        if prices[i] > prices[i-1]:
```




后续测试了oj的3道题目，均可以AC，且讲解清晰，感觉好强。

Q6. 阅读《Build a Large Language Model (From Scratch)》第一章

作者：Sebastian Raschka

1.1 什么是大语言模型（LLM）

- **定义：** LLM 是基于深度神经网络的模型，通过预训练大量文本数据，能够理解、生成和响应类人文本。
- **核心特点：**
 - **大规模参数：** 通常包含数亿至数千亿参数。
 - **自监督学习：** 通过预测文本中下一个词（next-word prediction）进行预训练。
 - **Transformer 架构：** 依赖自注意力机制（self-attention）处理长距离依赖和上下文关系。

1.2 LLM 的应用

- **多样化任务：** 文本生成、翻译、摘要、问答、代码编写、情感分析等。
- **实际场景：**
 - 聊天机器人（如 ChatGPT）、搜索引擎增强、医疗/法律知识检索、内容创作。
 - **关键能力：** 零样本（zero-shot）和少样本（few-shot）学习，无需任务特定训练。

1.3 构建与使用 LLM 的阶段

- 三阶段流程：
 1. **预训练 (Pretraining)**：在大规模无标签文本上训练基础模型（如 GPT-3）。
 2. **微调 (Fine-tuning)**：在特定任务数据集上优化模型：
 - **指令微调 (Instruction Fine-tuning)**：适配问答、翻译等任务。
 - **分类微调 (Classification Fine-tuning)**：适配垃圾邮件检测等分类任务。
 - **优势**：定制化模型在特定领域表现更优，且支持本地部署以提升隐私和效率。

1.4 Transformer 架构

- **核心组件**：
 - **编码器 (Encoder)**：处理输入文本，生成上下文编码。
 - **解码器 (Decoder)**：基于编码生成输出（如翻译结果）。
 - **自注意力机制**：动态加权输入词的重要性，捕捉全局依赖。
- **变体模型**：
 - **BERT** (双向编码器)：擅长分类任务（如情感分析）。
 - **GPT** (仅解码器)：专注于生成任务（如文本续写）。

1.5 大规模数据集的重要性

- **GPT-3 数据集**：
 - 包含 CommonCrawl、WebText2、Books 等来源，总计约 5000 亿词元。
 - **数据多样性**：覆盖多语言、多领域文本，使模型具备广泛知识。
- **训练成本**：GPT-3 预训练耗资约 460 万美元，凸显资源需求。

1.6 GPT 架构详解

- **核心设计**：
 - 基于 Transformer 解码器，无编码器。
 - **自回归 (Autoregressive)**：逐词生成，依赖上文预测下文。
 - **上下文窗口**：支持长文本连贯生成（如 GPT-3 的 2048 词窗口）。
- **关键改进**：
 - 层数增加（如 GPT-3 含 96 层）、参数规模扩展（1750 亿参数）。
 - **涌现能力**：通过简单任务（如词预测）间接掌握翻译、推理等复杂能力。

1.7 构建 LLM 的步骤

- **三阶段实现**：
 1. **架构与数据准备**：实现 Tokenizer、自注意力模块、模型结构。
 2. **预训练基础模型**：在大规模文本上训练词预测任务。
 3. **任务微调**：适配分类或指令遵循任务（如构建个人助手）。
- **教育意义**：通过简化模型（参数更少）理解核心机制，代码可在消费级硬件运行。

总结

LLM 的核心是通过 Transformer 架构和大规模预训练实现通用语言理解。其成功依赖于：

- **架构创新**：自注意力机制解决长序列建模问题。
- **数据驱动**：海量文本训练使模型捕捉语言规律。
- **两阶段训练**：预训练 + 微调平衡通用性与任务特异性。

2. 学习总结和个人收获

跟进每日选做, 同时学期初比较闲, 拓展阅读了很多我们物理领域与机器学习结合的文献